FC7xxx Mailbox Integration Manual

Rev.0.1

Revision History

Revision	Date	Changes
0.1	2023/10/20	Initial release for MCAL V0.3.0

Table of Contents

Chapter 1 Introduction 4
1.1 Intruduction 4
Chapter 2 Building 5
2.1 Dependencies on Other Modules 5
2.2 Files Required for Compile 5
2.3 Add Plug-ins 6
Chapter 3 Memory 7
3.1 Sections in Memory Map 7
Chapter 4 Exclusive Area 9
Chapter 5 Interrupt Service Routine (ISR) 10
Chapter 6 Error Report 11
6.1 Det 11
6.2 Dem 12
Chapter 7 Function Calls 13
7.1 Function Calls during Startup 13
7.2 Function Calls during Shutdown 13
7.3 Function Calls during Wake-up 13
7.4 Function Calls during Runtime 13
Chapter 8 Other Requirements 14
8.1 Calls to Notification Functions, Callbacks, Callouts 14
8.2 Macros 14
Chapter 9 Integration steps 15

Chapter 1 Introduction

1.1 Intruduction

This integration manual describes the integration requirements for the Mailbox (Mb) module.

Chapter 2 Building

2.1 Dependencies on Other Modules

Module configuration dependency

- Common: This module is the basic module which used to choose the chip.
- ECUC: This module provides the ECUC partition information for Mailbox module.

Module build dependency

- Common: This module provides common type for other modules.
- Rte: This module provides APIs to protect/unprotect some parts of code from interrupts (Exclusive Areas).
- Det: This module is necessary for enabling Development error detection.

2.2 Files Required for Compile

Mailbox module files:

- _MCAL/Src/Mb/src/CDD_Mb.c
- _MCAL/Src/Mb/src/CDD_Mb_Hw.c
- _MCAL/Src/Mb/src/CDD_Mb_Irq.c
- _MCAL/Src/Mb/include/CDD_Mb_Hw.h
- _MCAL/Src/Mb/include/CDD_Mb_Types.h
- _MCAL/Src/Mb/include/CDD_Mb_Version.h
- _MCAL/Src/Mb/include/Mb_MemMap.h

- _MCAL_generate/src/CDD_Mb_Cfg.c
- _MCAL_generate/src/CDD_Mb_PBcfg.c
- _MCAL_generate/include/CDD_Mb_Cfg.h

Common module files:

- _MCAL/Src/Common/include/Mcal.h
- _MCAL/Src/Common/include/Std_Types.h
- _MCAL/Src/Common/include/Platform_Types.h
- _MCAL/Src/Common/include/Compiler.h
- _MCAL/Src/Common/include/Compiler_Cfg.h
- _MCAL/Src/Common/include/CompilerDefinition.h
- _MCAL/Src/Common/include/Mb_Reg.h
- _MCAL/Src/Common/include/Mb_RegOps.h
- _MCAL/Src/Common/src/SpinLock.c
-

Det module files:

- Det.h
- Det.c

Rte module files:

- SchM_Mb.h
- SchM_Mb.c

2.3 Add Plug-ins

Mailbox module plug-ins are developed for EB tools, so, to use Mailbox plug-ins on EB tool, the user needs to add the Mailbox module to the EB plug-ins folder first.

1) Copy Mailbox module(_MCAL/EB_Plugins/eclipse/plugins/Mb) folder to EB tresos plug-ins (EB/tresos/plugins/) folder.
2) Set Mailbox module output location folder for the generated source file and header files.
3) Use EB tresos to configure Mailbox module and generate source and header files.

Chapter 3 Memory

3.1 Sections in Memory Map

Section Name	Section Type	Description
MB_START_SEC_CONFIG_DATA_8 MB_STOP_SEC_CONFIG_DATA_8 MB_START_SEC_CONFIG_DATA_16 MB_STOP_SEC_CONFIG_DATA_16 MB_START_SEC_CONFIG_DATA_32 MB_STOP_SEC_CONFIG_DATA_32	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by startup code (data).
MB_START_SEC_CONFIG_DATA_UNSPECIFIED MB_STOP_SEC_CONFIG_DATA_UNSPECIFIED	Configuration Data	Start and stop of Memory Section for Config Data.
MB_START_SEC_CONST_BOOLEAN MB_STOP_SEC_CONST_BOOLEAN MB_START_SEC_CONST_8 MB_STOP_SEC_CONST_8 MB_START_SEC_CONST_16 MB_STOP_SEC_CONST_16 MB_START_SEC_CONST_32 MB_STOP_SEC_CONST_32 MB_START_SEC_CONST_ UNSPECIFIED MB_STOP_SEC_CONST_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit or boolean. These variables are readonly (rodata).
MB_START_SEC_CODE MB_STOP_SEC_CODE MB_START_SEC_ RAMCODE MB_STOP_SEC_ RAMCODE MB_START_SEC_CODE_AC MB_STOP_SEC_CODE_AC	Code	Start and stop of memory Section for Code(text).
MB_START_SEC_VAR_NO_INIT_BOOLEAN MB_STOP_SEC_VAR_NO_INIT_BOOLEAN MB_START_SEC_VAR_NO_INIT_8 MB_STOP_SEC_VAR_NO_INIT_8 MB_START_SEC_VAR_NO_INIT_16 MB_STOP_SEC_VAR_NO_INIT_16 MB_START_SEC_VAR_NO_INIT_32 MB_STOP_SEC_VAR_NO_INIT_32 MB_START_SEC_VAR_NO_INIT_ UNSPECIFIED MB_STOP_SEC_VAR_NO_INIT_ UNSPECIFIED	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are never cleared and never initialized by startup code (bss).
MB_START_SEC_VAR_ INIT_BOOLEAN MB_STOP_SEC_VAR_ INIT_BOOLEAN MB_START_SEC_VAR_ INIT_8 MB_STOP_SEC_VAR_ INIT_8	Variables	These are all the sections used for variables which have to be aligned to 8/16/32 bit. These variables are initialized by

Section Name	Section Type	Description
MB_START_SEC_VAR _INIT_16 MB_STOP_SEC_VAR _INIT_16 MB_START_SEC_VAR _INIT_32 MB_STOP_SEC_VAR _INIT_32 MB_START_SEC_VAR _INIT_ UNSPECIFIED MB_STOP_SEC_VAR _INIT_ UNSPECIFIED		startup code (data).

Chapter 4 Exclusive Area

Mailbox module using the services of Scheduler Module (SchM) for entering and exiting critical regions.

The following critical regions are used in the Mailbox driver:

- CDD_Mb.c:
  - MB_EXCLUSIVE_AREA_00 Used in function Mb_SendMessage to protect the updates to:
    - Spin Lock
    - Message queue of the channel
    - Message queue ring block status of the channel
  - MB_EXCLUSIVE_AREA_01 Used in function Mb_GetMessage to protect the updates to:
    - Spin Lock
    - Message queue of the channel
    - Message queue ring block status of the channel
  - MB_EXCLUSIVE_AREA_02 Used in function Mb_GetMessageCount to protect the updates to:
    - Spin Lock
    - Message queue of the channel
    - Message queue ring block status of the channel
  - MB_EXCLUSIVE_AREA_03 Used in function Mb_GetMessageQueueState to protect the updates to:
    - Spin Lock
    - Message queue ring block status of the channel
  - MB_EXCLUSIVE_AREA_04 Used in function Mb_ResetMessageQueue to protect the updates to:
    - Spin Lock
    - Message queue ring block status of the channel
  - MB_EXCLUSIVE_AREA_05 Used in function Mb_ResetChannel to protect the updates to:
    - Mailbox hardware status
  - MB_EXCLUSIVE_AREA_06 Used in function Mb_GetChannelState to protect the updates to:
    - Mailbox hardware status

Chapter 5 Interrupt Service Routine (ISR)

Instance	Interrupt Name	IRQ Number (NVIC Interrupt ID)
Mailbox	MB_IRQHandler	51

Chapter 6 Error Report

6.1 Det

Function Name	Error Type
Mb_Init	MB_E_ALREADY_INITIALIZED MB_E_PARAM_CONFIG
Mb_DeInit	MB_E_UNINIT
Mb_SendData	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION MB_E_PARAM_CORE MB_E_CHANNEL_LOCKED
Mb_DoneChannel	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION MB_E_CHANNEL_UNLOCKED
Mb_SendMessage	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION MB_E_PARAM_CORE MB_E_PARAM_POINTER MB_E_PARAM_BUFFER_SIZE MB_E_CHANNEL_LOCKED MB_E_MESSAGE_QUEUE_FULL MB_E_GET_SPIN_LOCK_FAILED
Mb_GetMessage	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION MB_E_MESSAGE_QUEUE_EMPTY MB_E_GET_SPIN_LOCK_FAILED
Mb_GetMessageCount	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION MB_E_GET_SPIN_LOCK_FAILED
Mb_GetMessageQueueState	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION MB_E_GET_SPIN_LOCK_FAILED
Mb_ResetMessageQueue	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION

Function Name	Error Type
	MB_E_MESSAGE_QUEUE_BUSY MB_E_GET_SPIN_LOCK_FAILED
Mb_ResetChannel	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION MB_E_CHANNEL_UNLOCKED
Mb_GetChannelState	MB_E_UNINIT MB_E_PARAM_CHANNEL MB_E_INV_PARTITION
Mb_GetVersionInfo	MB_E_PARAM_VINFO

6.2 Dem

None

Chapter 7 Function Calls

7.1 Function Calls during Startup

The API needs to be called is Mb_Init(ConfigPtr);

7.2 Function Calls during Shutdown

None

7.3 Function Calls during Wake-up

None

7.4 Function Calls during Runtime

None

Chapter 8 Other Requirements

8.1 Calls to Notification Functions, Callbacks, Callouts

If the user configures notifications and the value is not "NULL_PTR" or "NULL", an extern declaration will generate in CDD_Mb_PBCfg.c. User need implement the notification in any file.

- Notification
  Mb_RequestNotification_<Channel> shoud be called if the request event occurs.
  Mb_DoneNotification_<Channel> shoud be called if the done event occurs.
  Mb_ReceivedNotification_<Channel> should be called if a channel configured as a message queue receives a message.
- Callback
  None
- Callout
- None

8.2 Macros

Please check various definitions available in Common module's include file Mcal.h for details.

- If OS is not used:
  AUTOSAR_OS_NOT_USED needs to be defined.

  In this case, If USE_SW_VECTOR_MODE is not defined, the user needs to map the interrupt vector table function to ISR function, for example:

```
extern ISR(MAILBOX_ISR);

void MB_IRQHandlerr(void)

{

MAILBOX_ISR ();

}
```

- If OS is used:
  Do not define AUTOSAR_OS_NOT_USED.

Chapter 9 Integration steps

1) Configure the Mailbox module and generate configuration files (please refer to Building chapter for details).
2) Configure appropriate memory sections in linker file or other (please refer to Memory chapter for details).
3) Map interrupt notification to their vector locations (please refer to chapter ISR for details).
4) Build the Mailbox module with dependent modules.